ClickFix First Aid: Five-area QA automation plan

This plan turns the architecture review into explicit, testable release criteria. **Scope limit:** the upload contains a 40-file bypass harness (30 HTML pages), not the extension source. The accompanying PDF reports application behavior. Accordingly, the attached unit suite is a fail-closed, real-source **contract suite** rather than a claim that the extension has passed these tests. The decision core and its JSON rules, page-world wrappers, notification channel, DOM analyzer, UI and settings need the actual source before their production implementations can be validated.

Use Node 20+ `node:test` with per-test mock functions and assertion failures (Node.js test runner).
DOM tests use jsdom; it does not implement browser layout or extension-world isolation, so the supplied Playwright pages remain mandatory integration coverage (jsdom README).
The adversarial strings are test fixtures, not commands to execute. The fixture tier uses mocked clipboard APIs and does not launch shells or fetch any command payload.

Detector and rules (`engine/detector.js`, `clickfix-rules.json`)

Risk: incorrect `allow`/`warn` decisions silently leave text on the clipboard. Unit target: `createDetector().decide(text, {strong,weak,subframe})`. Use real rules, not regexes copied into tests.

Class	Cases	Expected result
Happy path	Benign `winget`, `wsl`, Git comment, ordinary prose	`allow`; no accidental executor
Happy path	Strong CAPTCHA lure with `cmd` decoy; off-lure remote PowerShell	`block`/`hold`; warn-only off-lure is unacceptable
Obscure edge	En/em/other Unicode dashes, smart quotes, `/enc`, `-EncodedCom`, short base64	Normalize canonical syntax and retain protective verdict
Obscure edge	`%ComSpec%`, `%windir:~-7,1%`, 8.3 binary names, startup-folder curl, UNC WebDAV, macOS curl→chmod→execute	Recognize equivalent execution chains
Obscure edge	Six comment-only junk lines; `Array.prototype[0]` + `Object.prototype.lastIndex` accessors in a subprocess	Verdict unchanged; not downgraded by padding or realm pollution
Negative input	`null`, symbol, object; generic “verify identity” admin comments	Malformed inputs throw; benign comments do not trigger full-screen block

Additional boundary tests: Same corpus with subframe context; test local UNC installer vs remote UNC, format-operator invocations, different case-folding cultures, and max-length input. The PDF fixture JSON is clipped on page 15, so this suite uses **curated complete cases**, not a falsely reconstructed 14-case golden file. Add the real full fixtures when available.

Clipboard interception (`extension/page-hook.js`)

Risk: unhooked write path or check/use divergence bypasses replacement. Inject mocks for native `writeText`, `write`, `setData`, `items.add`, `clearData`, `execCommand` and `notify`; assert **actual native arguments and clipboard contents**, not only a UI verdict.

Class	Cases	Expected result
Happy path	Ordinary note via `writeText`; known decoy via `writeText`	Ordinary value intact; dangerous value replaced/held
Obscure edge	Object with changing `toString()`; `ClipboardItem` with unresolved `getType()` promise; HTML-only item	Convert once, inspect bytes written, invoke native while activation exists; do not invent plain text
Obscure edge	`items.add`, `setData`, `execCommand('copy')`, `clearData` then `items.add`	Same block/hold policy on all text/plain paths
Negative input	Native `NotAllowedError`, throwing `toString()`, non-plain-text format, multi-item write	Errors propagate and no false success event; unsupported forms explicit, no unexamined text/plain

Browser-only gates: Fresh and late `about:blank` iframe borrowed wrappers, `window.open`, cross-realm `setData`, closed-shadow textarea copy, user-activation window and real clipboard reads. The uploaded `tostring`, `itemsadd`, `iframe_*`, `shadow_ta`, `write_multi`, and `write_html_only` pages cover these paths. Unit mocks cannot establish Chromium's real clipboard behavior.

Notification channel (`page-hook.js notify`, `content.js` listener)

Risk: the reported DOM CustomEvent is forgeable, interceptable, and globally throttled. After hardening, inject `authenticate`, `recompute`, `show`, and a monotonic clock. The source adapter must delegate to the actual message receiver, not simply accept a page field that claims it is trusted.

Class	Cases	Expected result
Happy path	Authenticated report with trusted-side recomputation	One genuine UI report; not blind trust of claimed verdict
Obscure edge	`allow` then block at 1 ms; warn then block at 1 ms; 799/800 ms boundary	Only same-verdict throttle; stronger alert not suppressed

Class	Cases	Expected result
Obscure edge	Duplicate report id; cross-frame claimed block vs local `allow`	No duplicate counter; no unverified top-frame overlay
Negative input	Forged `trusted:true`, malformed JSON, oversized text, poisoned accessor, invalid method, detector exception	No overlay, no counter increment, no unsafe getter call

Browser-only gates: Create a hostile page listener with `stopImmediatePropagation`; dispatch a forged event in a cross-origin ad frame; check MAIN-before-page and isolated listener startup order; confirm a real block still shows UI. Use `forged`, `forged_frame`, `flood`, and `stopprop`.

Page-context analysis (`content.js analyzeThisPage`)

Risk: attacker text is hidden at capture time while ordinary tutorial text triggers a false block. Unit tests inject `document` and `getComputedStyle` mocks and exercise cache invalidation. The classifier returns `weak`, `strong`, bounded `text`, and a `truncated` signal.

Class	Cases	Expected result
Happy path	Visible fake verification + Win+R/Ctrl+V/Enter; ordinary prose	Strong true; ordinary prose false
Obscure edge	Open shadow roots; CSS `::before`; hidden filler between visible words; late DOM mutation and cache invalidation	Strong detection on visible cues and refreshed result after mutation
Obscure edge	210k-character page; empty document	Bounded extraction and honest truncation; no crash
Negative input	Script/style/template/noscript/textarea-only text; normal Win+R tutorial	Neither secret/JSON text nor weak tutorial becomes strong lure
Negative input	Closed shadow root or cross-origin iframe	No misleading claim it can read inaccessible text; no exception

Browser-only gates: Computed `::before` text, `innerText` layout behavior, hidden filler and cross-origin iframe steps; late mutation *after* a clipboard write; large DOM copy-path latency. Verify both false negatives and false positives with `split_junk`, `css_content2`, `shadow`, `xorigin_steps`, `late_instr2`, `fp_tutorial`, `fp_nextdata`. Closed roots and inaccessible frames remain blind spots: protect remote executors independently rather than promising to parse inaccessible DOM.

UI state machine and settings (`ui.js`, `content.js`)

Risk: warning toast prevents lure banner, settings overwrite each other, and attacker-supplied preview content can look like trusted instructions. Inject `render`, scheduler, storage, and document; assert state **after** rendering succeeds, not before.

Class	Cases	Expected result
Happy path	Warning → 15-second expiry; soft-warning toggle	Toast removed at TTL; unrelated setting keys preserved
Obscure edge	warn→lure→block; block→warn→lure; pending warn render vs block	Stronger state wins; stale timer cannot erase banner/block
Obscure edge	Two tabs mute different sites concurrently; popup toggles while old tab mutes	Atomic merge; no lost site or resurrected toggle
Negative input	Render throws; storage update fails; invalid URL; malformed event	No permanent flag on failed mount, no false “saved”, explicit errors
Negative input	HTML/script tag or phone/URL in attacker-controlled preview	Render as inert text, label as page-supplied; never trusted instructions

Browser-only gates: Inspect real closed-shadow UI rather than a mock renderer; verify warning does not suppress later banner; verify multi-tab storage events and cross-origin spoofing. Source access is necessary to bind a `previewElement()` accessor to the production element for DOM assertions.

Exit criteria and execution status

1. Fixture tier passes: source pages really encode the intended harmless scenarios. All 11 fixture checks passed locally. Passing them is **not** passing the application.
2. Adapter preflight succeeds against a source checkout and imports the real implementation of all five areas (see `qa-suite/CONTRACT.md`). At present preflight exits 1 with an explicit source-absent error.
3. All 53 app-facing contract tests pass with no skipped/relaxed security assertions; each known v0.1 failure is fixed or documented as a release blocker.
4. Run Chromium integration suites against the unpacked extension on both false-negative and false-positive pages and record actual clipboard plus UI state. Do not claim these checks until run.
5. Windows checker scripts are outside these five but still need a separate Windows-host regression suite before shipping.

Correction to the earlier structural summary: It described the CustomEvent as if it necessarily carried the synchronous verdict request between worlds. The PDF specifically identifies `notify()` dispatching events and a `content.js` listener; without the extension source the complete call order is not established. The test plan therefore targets

notifications and explicitly requires source-level binding to confirm where `decide()` runs.